



S.G. Ganesh

Test Your C skills

In this column, we'll look at how some macros are implemented in C header files in Linux.

I've given dummy names for some #defines from a few C header files from GCC (Linux) implementation. By looking at the definition of the macro, guess the name of the macro and which header file it is from.

- 1) #define MACRO1 (-INT_MAX - 1)
#define MACRO2 (INT_MAX * 2U + 1)
- 2) #define MACRO3(c) ((unsigned)(c)<=0177)
#define MACRO4(c) ((c)&0177)
- 3) #define MACRO5() getc(stdin)
#define MACRO6(x) putc(x, stdout)
- 4) #define MACRO7 ((void *)0)
#define MACRO8(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)

Now, check your answers.

1. From <limits.h>: MACRO1 is INT_MIN and MACRO2 is UINT_MAX. You can cross-check if the values are correct with an example: If size of int is 2, range is -32768 to 32767 and 65535 is equal to ((32767 * 2) + 1).
An aside: Some implementations define INT_MAX as (((unsigned int)~0)>>1), ~0 is -1 (all 1s in binary is decimal value -1 in signed integer). When -1 is casted to unsigned int, it is UINT_MAX. If you right-shift it by one, it is INT_MAX in int.
2. From <ctype.h>: MACRO3 is isascii and MACRO4 is toascii. In the octal value 0177, 1 is 001 and 7 is 111 in binary. So, 0177 fills 1s in a byte except for the sign-bit, i.e., it stands for binary value 01111111 (which is 127 in decimal and 0x7F in hexa). ASCII values are from 0 to 127. Given the argument 'c', the second macro takes only the lowest byte (by 'and'ing it with all ones for 7 bits); so it implements toascii.
3. From <stdio.h>: MACRO5 is getchar and MACRO6 is putchar. getchar and putchar are short-cuts for calling the getc and putc with stdin and stdout respectively (most C programmers think that getchar and putchar are separate functions!).
4. From <stddef.h>: MACRO7 is NULL and MACRO8 is offsetof.

In C, NULL is used universally for initialising all pointer types, for example, "int *p = NULL". We can define NULL as 0, but 0 is an integer value—it is better to treat it as a pointer value. So, we convert it explicitly to the universal pointer type (void *), and hence this macro definition.

The offset of a macro is a little difficult to understand, so, I'll explain in detail. This macro is

used for finding the offset of a struct member from the base address of the struct variable (in other words, the number of bytes from the base address of the struct variable). For example, consider a "struct student" that has members like "name", "DOB" (date-of-birth), "rollno", etc.:

```
struct student {
    char name[20];
    struct {
        short day; short month; short year;
    } DOB;
    int rollno;
};
```

Now, how many bytes is "rollno" from the base address of the struct? (Hint: We need to consider padding also, and it differs from machine to machine; for example, the offset of "rollno" is 28 and not 26 in my machine.) To find that (portably), we should use "offsetof" as in:

```
size_t position = offsetof(struct student, rollno);
```

To understand how this implementation works, let's look at the expansion first:

```
size_t position = ((size_t) &((student *)0)->rollno)
```

We'll start from the innermost expression: "((student *)0)". This is to get an expression of type "(student *)" for the base address 0. Now, "((student *)0)->rollno" refers to the member "rollno". After that, using "&" (addressof) takes the address of the "rollno" member in "student struct". Since the base address is 0, we're getting the number of bytes from which "rollno" is from 0 (which is also the base address of the struct variable). Since the type of the resulting value is expected as "size_t", we cast the expression as "(size_t)".

Now a bonus: Here is an alternative implementation of "offsetof", based on same logic:

```
#define offsetof(type, mem) ((size_t) ((char *)&((type *)0)->mem - (char *)0))
```

Okay, that's all for this month: Go ahead and explore more such macros in C header files! 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.